2. Lenguajes de programación

2.1 Concepto

Un **lenguaje de programación** es un sistema de símbolos y reglas que permite dar instrucciones a una computadora.

Es como un “idioma” para comunicarse con ella.

2.2 Características principales

- **Sintaxis**: reglas para escribir correctamente el código.
- **Semántica**: significado de las instrucciones.
- **Precisión**: no hay ambigüedades.
- **Estructura**: permite organizar el código (funciones, clases…).
- **Portabilidad**: funciona en distintos sistemas (en algunos casos).
- **Nivel de abstracción**:
  - Bajo nivel → más cerca del hardware
  - Alto nivel → más fácil para humanos
- **Eficiencia**: uso de recursos y velocidad.

2.3 Elementos de los lenguajes de programación por niveles

Nivel léxico (léxico)

Elementos básicos del lenguaje (tokens):

- **Palabras clave**: `if`, `while`, `int`
- **Identificadores**: nombres de variables (`edad`, `total`)
- **Literales**: valores (`10`, `"hola"`, `true`)
- **Operadores**: `+`, `-`, `=`, `==`
- **Separadores**: `;`, `,`, `{ }`, `( )`

Son las “piezas” mínimas del código.

Nivel sintáctico (sintaxis)

Elementos que organizan el código:

- **Expresiones**: `a + b`
- **Sentencias**: `int x = 5;`
- **Bloques de código**: `{ ... }`
- **Estructuras de control**: `if`, `for`, `while`
- **Declaraciones**: variables, funciones, clases

Define cómo se combinan los tokens.

Nivel semántico (semántica)

Elementos relacionados con el significado:

- **Tipos de datos**: `int`, `float`, `String`
- **Compatibilidad de tipos**: operaciones válidas o no
- **Ámbito (scope)**: dónde existe una variable
- **Inicialización de variables**
- **Significado de operaciones**: qué hace realmente cada instrucción

Asegura que el programa tenga sentido lógico.

2.4 Instrucciones

Las instrucciones en programación son órdenes que se dan al ordenador para ejecutar acciones concretas. Se pueden clasificar en:

a). Instrucciones de declaración

Sirven para crear variables o constantes.

Ejemplo en Java:

```
int edad = 20;
```

b) Instrucciones de asignación

Sirven para modificar el valor de una variable.

```
edad = 25;
```

c) Instrucciones de entrada/salida

Permiten mostrar información o recibir datos.

```
System.out.println("Hola");
```

d) Instrucciones de control de flujo

Condicionales:

```
if (edad >= 18) {
    System.out.println("Adulto");
}
```

Bucles:

```
while (edad < 30) {
    edad++;
}
```

e) Instrucciones de salto

Modifican el flujo del programa:

- break → sale de un bucle
- continue → salta una iteración
- return → termina una función

3. Tipos de lenguajes de programación

3.1 Clasificación por nivel de abstracción

La clasificación por nivel de abstracción indica qué tan cerca está un lenguaje del hardware o del lenguaje humano.

a) Bajo nivel (poca abstracción)

Son los más cercanos al hardware.

- Lenguaje máquina (binario: 0 y 1)
- Lenguaje ensamblador

Características:

- Muy difíciles de entender
- Muy rápidos y eficientes
- Dependientes del hardware

b) Nivel medio

Actúan como puente entre hardware y software.

Ejemplo:

- C

Características:

- Más fáciles que los de bajo nivel
- Permiten control de memoria

c) Alto nivel (alta abstracción)

Más cercanos al lenguaje humano.

Ejemplos:

- Python
- Java
- C#

Características:

- Fáciles de aprender y usar
- Independientes del hardware
- No requieren gestión directa de memoria

d). Muy alto nivel (ultra abstracción)

Se centran en qué hacer, no cómo hacerlo.

Ejemplos:

- SQL
- Prolog

Características:

- Muy expresivos
- El sistema resuelve la lógica internamente

3.2 Clasificación cronológica

1ª generación (1GL) – Lenguaje máquina

- Basado en código binario (0 y 1)
- Ejecutado directamente por el hardware
- Muy difícil de programar

Ejemplo: 10110000 01100001

2ª generación (2GL) – Lenguaje ensamblador

- Usa mnemónicos en lugar de binario
- Más legible que el lenguaje máquina
- Dependiente del hardware

Ejemplo (Assembly): MOV AX, 1 ADD AX, 2

3ª generación (3GL) – Lenguajes de alto nivel

- Más cercanos al lenguaje humano
- Independientes del hardware
- Uso de estructuras como bucles y funciones

Ejemplos:

- C
- Java

- Python

4ª generación (4GL) – Lenguajes orientados a problemas

- Más declarativos (qué hacer, no cómo hacerlo)
- Usados en bases de datos y análisis

Ejemplos:

- SQL
- MATLAB
- R

5ª generación (5GL) – Inteligencia artificial

- Basados en lógica y resolución automática de problemas
- Usados en IA y sistemas expertos

Ejemplos:

- Prolog

3.3 Clasificación por Paradigmas

Un paradigma de programación representa un enfoque filosófico o un modelo para la resolución de problemas mediante código. A continuación, se detallan las categorías principales:

a) Paradigma Imperativo

Se centra en describir **cómo** debe funcionar el programa mediante cambios de estado y secuencias de comandos.

- **Programación Estructurada:** Utiliza estructuras de control (bucles, condicionales) para hacer el código legible y evitar saltos arbitrarios.
  - *Ejemplos:* **C, Pascal, ALGOL.**
- **Programación Orientada a Objetos (POO):** Organiza el software en "objetos" que agrupan datos (atributos) y comportamientos (métodos). Se basa en conceptos como herencia, encapsulamiento y polimorfismo.
  - *Ejemplos:* **Java, C++, Ruby, C#.**
- **Programación Procedural:** Basada en llamadas a procedimientos o funciones que operan sobre datos.
  - *Ejemplos:* **Fortran, COBOL.**

b) Paradigma Declarativo

Se centra en describir **qué** es lo que se desea obtener o qué problema se quiere resolver, sin detallar los pasos exactos del flujo de control.

- **Programación Funcional:** Trata la computación como la evaluación de funciones matemáticas, evitando el cambio de estado y los datos mutables.
  - *Ejemplos:* **Haskell, Lisp, Elixir, Scala.**

- **Programación Lógica:** Basada en la lógica matemática. El programador define reglas y hechos, y el sistema deduce las respuestas.
    - *Ejemplos:* **Prolog.**
- **Lenguajes de Consulta:** Diseñados específicamente para interactuar con bases de datos o estructuras de datos complejas.
    - *Ejemplos:* **SQL, XQuery.**

c) Paradigmas Especializados

- **Programación Reactiva:** Orientada al manejo de flujos de datos asíncronos y la propagación de cambios.
    - *Ejemplos:* **RxJS, Elm.**
- **Programación Orientada a Aspectos (POA):** Permite separar funcionalidades transversales (como seguridad o logs) del núcleo del programa.

d) Lenguajes Multiparadigma

En la actualidad, la mayoría de los lenguajes populares no pertenecen a una sola categoría, sino que permiten combinar estilos según la necesidad.

Lenguaje	Paradigmas principales que soporta
Python	Imperativo, POO, Funcional.
JavaScript	Funcional, Basado en prototipos (POO), Imperativo.
Rust	Imperativo, Funcional, Genérico.
Swift	POO, Funcional, Orientado a protocolos.

Nota: El uso de un paradigma u otro depende del tipo de proyecto (ej. funcional para análisis de datos masivos, POO para grandes sistemas empresariales).

3.4 Otras clasificiiones

a) Según el lugar de ejecución:

Ejecución en el Lado del Cliente (Client-side): El código se descarga desde un servidor, pero se ejecuta directamente en el dispositivo del usuario (computadora, tablet o smartphone).

- Cómo funciona: El procesador local y la memoria RAM del usuario son los que realizan el trabajo.
- Lenguaje Rey: JavaScript (y sus derivados como TypeScript). También lenguajes que compilan a WebAssembly (como Rust o C++ ejecutándose en el navegador).
- Ventajas:
    - Respuesta inmediata (no hay que esperar al servidor).
    - Interactividad fluida (animaciones, validación de formularios al instante).
    - Ahorra costes de procesamiento al dueño de la web.

- Desventajas: Depende de la potencia del dispositivo del usuario y el código es visible para cualquiera ("Ver código fuente").

Ejecución en el Lado del Servidor (Server-side): El código se ejecuta en una computadora remota (servidor). El usuario solo recibe el resultado final, generalmente en forma de HTML, JSON o una imagen.

- Cómo funciona: El usuario envía una petición, el servidor procesa la lógica, consulta bases de datos y devuelve solo la respuesta procesada.
- Lenguajes comunes: Python, PHP, Java, Go, Ruby, C#, Node.js.
- Ventajas:
    - Seguridad: La lógica de negocio y las bases de datos están protegidas; el usuario no ve el código.
    - Consistencia: No importa si el usuario tiene un móvil viejo o una PC potente; el servidor hace el trabajo igual de rápido.
- Desventajas: Requiere una conexión a internet constante y cada acción genera una pequeña espera (latencia) mientras viajan los datos.

Ejecución en el Borde (Edge Computing): Es la tendencia más moderna. El código no se ejecuta ni "tan lejos" como el servidor central, ni "tan cerca" como el dispositivo del usuario.

- Cómo funciona: Se ejecuta en servidores intermedios distribuidos geográficamente (nodos de una red como Cloudflare o AWS Lambda@Edge), muy cerca de la ubicación física del usuario.
- Propósito: Reducir la latencia al mínimo absoluto para aplicaciones globales.

b) Según el método de ejecución

Lenguajes Compilados: En estos lenguajes, un programa llamado compilador traduce todo el código fuente de una sola vez y genera un archivo ejecutable (como un .exe o un binario de Linux).

- Proceso: Código Fuente → Compilador → Código Máquina (Binario).
- Ventajas: Son extremadamente rápidos y eficientes, ya que la traducción se hace una sola vez antes de ejecutar.
- Desventajas: Si haces un cambio, debes volver a compilar todo. No son multiplataforma por defecto (un binario de Windows no sirve en Mac).
- Ejemplos: C, C++, Rust, Go, Pascal.

Lenguajes Interpretados: Aquí no hay un archivo ejecutable previo. Un programa llamado intérprete lee el código línea por línea y lo ejecuta en tiempo real.

- Proceso: Código Fuente → Intérprete → Ejecución inmediata.
- Ventajas: Facilitan el desarrollo y la depuración. Son altamente portátiles: el mismo código funciona en cualquier sistema que tenga el intérprete instalado.

- Desventajas: Son más lentos que los compilados, ya que la máquina debe traducir el código mientras lo ejecuta.
- Ejemplos: Python, Ruby, JavaScript (en su concepción original), PHP.

Lenguajes Híbridos (Intermedios): Son una mezcla de ambos mundos. El código fuente se compila primero a un lenguaje intermedio (llamado Bytecode) y luego una Máquina Virtual lo interpreta o lo compila "al vuelo" (JIT - Just In Time) en el dispositivo del usuario.

- Proceso: Código Fuente → Compilador → Bytecode → Máquina Virtual → Código Máquina.
- Ventajas: Combinan la portabilidad de los interpretados con una velocidad cercana a los compilados.
- Ejemplos: Java (usa la JVM), C# (usa .NET), Kotlin.

Compilación JIT (Just-In-Time): Es una técnica moderna utilizada principalmente por lenguajes interpretados o híbridos para ganar velocidad. En lugar de interpretar cada línea siempre, el motor identifica las partes del código que se usan mucho y las compila a código máquina real mientras el programa corre.

- Ejemplo estrella: JavaScript moderno (motores como V8 de Chrome) y Julia.

4. Caracteristicas de los lenguajes de programación

A continuación señalamos los lenguajes de programación más influyentes de la historia, sus características principales y su impacto en la tecnología actual.

a). Los Pioneros: Sentando las Bases (1950 - 1960)

Fortran (1957)

El "abuelo" de los lenguajes de alto nivel. Fue diseñado para facilitar el trabajo de científicos e ingenieros.

- **Características:** Optimizado para cálculos numéricos y científicos complejos.
- **Legado:** Sigue siendo utilizado hoy en día en supercomputación y modelos climáticos.

Lisp (1958)

Introdujo conceptos revolucionarios como la recursividad y las funciones como ciudadanos de primera clase.

- **Características:** Basado completamente en listas y una sintaxis llena de paréntesis.
- **Legado:** Pionero absoluto en el campo de la **Inteligencia Artificial**.

COBOL (1959)

Diseñado para ser un lenguaje universal para el mundo de los negocios.

- **Características:** Sintaxis muy verbosa y similar al inglés para que fuera legible por ejecutivos.
- **Legado:** Gestiona todavía hoy la gran mayoría de los sistemas bancarios y transacciones financieras globales.

BASIC (1964)

Creado para que estudiantes que no eran de ciencias pudieran interactuar con las computadoras.

- **Características:** Extremadamente sencillo de aprender, basado en números de línea.
- **Legado:** Fue el lenguaje que impulsó la era de las microcomputadoras (Microsoft nació creando una versión de BASIC).

b). Los Arquitectos del Software (1970 - 1980)

Pascal (1970)

Diseñado por Niklaus Wirth con un objetivo académico: enseñar buenas prácticas de programación.

- **Características:** Estructura rígida y clara que obliga a una programación ordenada.
- **Legado:** Fue el estándar educativo en las universidades durante décadas.

C (1972)

Probablemente el lenguaje más influyente jamás creado. Es la base de los sistemas operativos modernos.

- **Características:** Bajo nivel (acceso directo al hardware) pero con portabilidad. Extremadamente rápido.
- **Legado:** Unix, Linux y Windows están escritos en C. Es la base sintáctica de Java, C++, PHP y JavaScript.

C++ (1983)

Una extensión de C que introdujo la **Programación Orientada a Objetos (POO)**.

- **Características:** Combina la velocidad de C con la capacidad de organizar grandes sistemas mediante objetos.
- **Legado:** Motores de videojuegos (Unreal Engine), navegadores web y software de alto rendimiento como Photoshop.

c). La Revolución del Internet y la Productividad (1990)

Python (1991)

Creado bajo la filosofía de que "la legibilidad cuenta".

- **Características:** Sintaxis limpia que parece inglés. Es multiparadigma (orientado a objetos, funcional).
- **Legado:** El lenguaje más popular hoy para **Ciencia de Datos, IA** y automatización.

R (1993)

El lenguaje especializado para el análisis estadístico.

- **Características:** Entorno de software libre con herramientas potentes para gráficos y minería de datos.
- **Legado:** El estándar de oro en la academia y la estadística avanzada.

Java (1995)

Su promesa fue: "Write Once, Run Anywhere" (Escríbelo una vez, ejecútalo en cualquier lugar).

- **Características:** Se ejecuta sobre una Máquina Virtual (JVM), lo que lo hace muy seguro y portátil.
- **Legado:** El corazón de las aplicaciones empresariales bancarias y de Android.

JavaScript (1995)

Creado originalmente en 10 días para añadir interactividad básica a las páginas web.

- **Características:** Es el único lenguaje nativo de los navegadores web.
- **Legado:** Hoy es ubicuo gracias a Node.js, permitiendo desarrollar tanto el frente (front-end) como el servidor (back-end).

d). La Era Moderna: Rendimiento, Nube y Seguridad (2000 - Presente)

Go / Golang (2009)

Diseñado por Google para resolver problemas de escalabilidad y sistemas distribuidos.

- **Características:** Muy simple y extremadamente eficiente en **concurrencia** (manejar múltiples tareas a la vez).
- **Uso notable: Podman**, Docker y Kubernetes están escritos principalmente en **Go**.

Rust (2010)

Enfocado en la seguridad y el rendimiento.

- **Características:** Garantiza la **seguridad de memoria** sin necesidad de un recolector de basura, evitando errores que causan hackeos o caídas.
- **Legado:** Está empezando a reemplazar a C++ en sistemas críticos y núcleos de sistemas operativos.

Kotlin (2011)

La alternativa moderna de JetBrains para mejorar Java.

- **Características:** Mucho más conciso y seguro que Java, pero totalmente compatible con él.
- **Legado:** Lenguaje oficial para el desarrollo de **Android**.

Julia (2012)

Diseñado para la computación científica de alto nivel.

- **Características:** Resuelve el "problema de los dos lenguajes": facilidad de Python con velocidad de C.
- **Legado:** Utilizado en investigación avanzada, física y finanzas.

5. Lenguajes más utilizados

El Top 10 del índice TIOBE a abril de 2026:

Puesto	Lenguaje
1	Python
2	C
3	C++
4	Java
5	C#
6	JavaScript
7	Visual Basic
8	SQL
9	R
10	Delphi / Pascal

6. Conclusión

La evolución de los lenguajes de programación representa una de las mayores proezas del ingenio humano: la capacidad de traducir el pensamiento lógico en realidades tangibles.

Los lenguajes de programación no son solo herramientas técnicas; son el puente entre la creatividad humana y el potencial de las máquinas. A lo largo de la historia, han pasado de ser crípticos y mecánicos a ser fluidos y casi naturales, democratizando el acceso a la creación de tecnología.

Beneficios Clave:

- Automatización y Eficiencia: Han permitido que tareas que antes tomaban meses se realicen en milisegundos, liberando al ser humano de trabajos repetitivos.
- Resolución de Problemas Complejos: Desde descifrar el genoma humano hasta modelar el cambio climático, la programación permite procesar volúmenes de datos inalcanzables para la mente biológica.
- Conectividad Global: El desarrollo de la infraestructura de internet y las comunicaciones modernas es, en su esencia, una red de protocolos escritos en diversos lenguajes.
- Innovación Constante: Cada nuevo lenguaje (como Rust o Go) surge para resolver una deficiencia del anterior, garantizando que el software sea cada vez más seguro, rápido y accesible.

En definitiva, programar es el superpoder de nuestra era. Los lenguajes de programación han transformado el mundo en un lienzo digital donde el único límite para construir el futuro es la imaginación y la lógica del desarrollador.

Generar PDF